

Phase 3 Documentation

Formal Proof in Lean 4

Chosen Theorem: Expression Determinism

Item	Description
Project phase	Phase 3 - Formal Proof
Proof assistant	Lean 4
Chosen theorem	Expression Determinism
Main deliverable	A self-contained proof.lean file that compiles successfully
Core claim	The same expression evaluated in the same state cannot produce two different values.

Document purpose

This document explains the design idea, formal model, and proof structure of the Lean 4 implementation for Phase 3. It is written as a presentation-ready companion to the proof.lean file.

Contents

- 1. Executive Summary
- 2. Theorem Selection and Formal Meaning
- 3. Design Overview
- 4. Codebase Walkthrough
- 5. Proof Strategy
- 6. Statement Semantics Included for Completeness
- 7. Compilation and Expected Result
- 8. Limitations and Possible Extensions
- Appendix A. Full Lean 4 Source Code

1. Executive Summary

Phase 3 of the project moves from model checking a concrete Promela model to proving a mathematical property about the Hash language subset itself. The implementation is written in Lean 4 and focuses on the selected theorem Expression Determinism.

The proof states that expression evaluation is deterministic: if an expression e is evaluated in the same program state s twice, and the first derivation produces $v1$ while the second produces $v2$, then $v1$ and $v2$ must be equal.

The Lean file is self-contained. It defines variable names, program states, expression syntax, an executable expression evaluator, relational big-step expression evaluation rules, the main proof, and a small statement-level operational semantics model for completeness.

2. Theorem Selection and Formal Meaning

The selected theorem is Expression Determinism. In mathematical notation, the intended property is:

$$\langle e, s \rangle \Downarrow v1 \text{ and } \langle e, s \rangle \Downarrow v2 \text{ implies } v1 = v2$$

In Lean, this is encoded using the relation `EvalExpr`. The theorem is expressed as:

```
theorem expression_determinism :
  forall {e : Expr} {s : State} {v1 v2 : Int},
    EvalExpr e s v1 -> EvalExpr e s v2 -> v1 = v2 := by
```

The meaning is direct: for every expression e , state s , and integer values $v1$ and $v2$, if there are two valid evaluation derivations for the same expression in the same state, both results must be equal.

3. Design Overview

The implementation follows a layered design. Each layer introduces a small formal component and the next layer builds on it.

Item	Description
Var	Variable names are modeled as strings.
State	A program state is a function from variable names to integer values.
Expr	Expressions are represented by an inductive datatype.
evalExpr	A deterministic executable evaluator used as a reference meaning.
EvalExpr	A relational big-step semantics for expression evaluation.
evalExpr_sound	A theorem showing that every relational evaluation agrees with evalExpr.
expression_determinism	The final theorem proving that expression evaluation is deterministic.
Stmt and BigStep	Statement syntax and statement semantics included to complete the operational-semantics model.

Key design idea

Instead of proving determinism directly by comparing two `EvalExpr` derivations case by case, the proof first connects every derivation to the deterministic function `evalExpr`. Since both derivations must equal the same function result, they must equal each other.

4. Codebase Walkthrough

4.1 Namespace

```
namespace ExpressionDeterminismProof
...
end ExpressionDeterminismProof
```

The namespace groups all definitions under `ExpressionDeterminismProof`. This prevents name conflicts with other files or earlier exercises. It is especially useful when multiple Lean files define names such as `Expr`, `State`, or `BigStep`.

4.2 Variables and States

```
abbrev Var := String
abbrev State := Var -> Int
```

Var is an abbreviation for String. A variable name such as `x` or `sum` is represented by a string value such as `"x"` or `"sum"`.

State is modeled as a function from Var to Int. In other words, a state is a memory map: given a variable name, it returns that variable's integer value. This representation is simple and avoids needing an explicit dictionary structure.

For example, if `s` is a state, then `s "x"` means the value of variable `x` in state `s`.

4.3 Expression Syntax

```
inductive Expr where
| num : Int -> Expr
| var : Var -> Expr
| add : Expr -> Expr -> Expr
| mul : Expr -> Expr -> Expr
| lt : Expr -> Expr -> Expr
| eq : Expr -> Expr -> Expr
deriving Repr
```

Expr defines the abstract syntax of expressions in the selected Hash subset. The constructors correspond to the expression forms supported by the formal semantics.

Item	Description
Expr.num n	A numeric literal, such as 5 or -2.
Expr.var x	A variable expression, such as x.
Expr.add e1 e2	Addition of two subexpressions.
Expr.mul e1 e2	Multiplication of two subexpressions.
Expr.lt e1 e2	Less-than comparison of two subexpressions.
Expr.eq e1 e2	Equality comparison of two subexpressions.
deriving Repr	Allows Lean to display values of this type in evaluation output if needed.

Boolean results are represented as integers: true is encoded as 1 and false is encoded as 0. This keeps expression values uniformly typed as Int and avoids introducing a separate Value datatype.

4.4 Executable Evaluator: evalExpr

```
def evalExpr (e : Expr) (s : State) : Int :=
  match e with
  | Expr.num n => n
  | Expr.var x => s x
  | Expr.add e1 e2 => evalExpr e1 s + evalExpr e2 s
  | Expr.mul e1 e2 => evalExpr e1 s * evalExpr e2 s
  | Expr.lt e1 e2 =>
    if evalExpr e1 s < evalExpr e2 s then 1 else 0
  | Expr.eq e1 e2 =>
    if evalExpr e1 s = evalExpr e2 s then 1 else 0
```

evalExpr is a recursive deterministic function. It takes an expression and a state, then returns the integer value of the expression in that state.

The num case returns the literal value. The var case reads the variable from the state. The add and mul cases recursively evaluate both subexpressions and combine the results. The lt and eq cases recursively evaluate both sides and return 1 or 0 depending on the comparison result.

This function is not itself the relational semantics. It is used as a deterministic reference meaning to simplify the proof of expression determinism.

4.5 Relational Expression Semantics: EvalExpr

EvalExpr is the formal big-step relation for expression evaluation. The proposition `EvalExpr e s v` means: expression `e` evaluates to value `v` in state `s`.

```
inductive EvalExpr : Expr -> State -> Int -> Prop where
```

Unlike `evalExpr`, which computes a value, `EvalExpr` describes valid evaluation derivations as logical evidence. Each constructor corresponds to an inference rule from the operational semantics.

4.5.1 Numeric Literals and Variables

```
| num : forall (n : Int) (s : State),
  EvalExpr (Expr.num n) s n

| var : forall (x : Var) (s : State),
  EvalExpr (Expr.var x) s (s x)
```

The `num` rule says that a numeric literal evaluates to itself in any state. The `var` rule says that a variable evaluates to the value stored for that variable in the current state.

4.5.2 Addition and Multiplication

```
| add : forall (e1 e2 : Expr) (s : State) (v1 v2 : Int),
  EvalExpr e1 s v1 ->
  EvalExpr e2 s v2 ->
  EvalExpr (Expr.add e1 e2) s (v1 + v2)

| mul : forall (e1 e2 : Expr) (s : State) (v1 v2 : Int),
  EvalExpr e1 s v1 ->
  EvalExpr e2 s v2 ->
  EvalExpr (Expr.mul e1 e2) s (v1 * v2)
```

The `add` and `mul` rules are compositional. To evaluate a compound expression, Lean first requires evidence that both subexpressions evaluate to some values `v1` and `v2`. The result is then constructed from those two values.

4.5.3 Comparisons

```
| lt : forall (e1 e2 : Expr) (s : State) (v1 v2 : Int),
  EvalExpr e1 s v1 ->
  EvalExpr e2 s v2 ->
  EvalExpr (Expr.lt e1 e2) s (if v1 < v2 then 1 else 0)

| eq : forall (e1 e2 : Expr) (s : State) (v1 v2 : Int),
  EvalExpr e1 s v1 ->
  EvalExpr e2 s v2 ->
  EvalExpr (Expr.eq e1 e2) s (if v1 = v2 then 1 else 0)
```

The `lt` and `eq` rules evaluate both sides first, then convert the comparison result into an integer. This is consistent with the design convention that `true` is 1 and `false` is 0.

5. Proof Strategy

The proof is divided into two steps. First, the theorem `evalExpr_sound` proves that every relational evaluation agrees with the deterministic function `evalExpr`. Second, `expression_determinism` uses that agreement to show that two relational evaluations of the same expression must produce the same value.

5.1 Soundness of Relational Evaluation

```
theorem evalExpr_sound :
  forall {e : Expr} {s : State} {v : Int},
    EvalExpr e s v -> v = evalExpr e s := by
```

This theorem states that if `EvalExpr e s v` holds, then `v` is exactly the value returned by `evalExpr e s`. In other words, the relational rules are sound with respect to the executable evaluator.

The proof begins by introducing the variables and the proof object `h`:

```
intro e s v h
induction h with
```

The induction is performed on `h`, the derivation of `EvalExpr e s v`. This is natural because `EvalExpr` is inductively defined. Each case of the proof corresponds to one constructor of `EvalExpr`.

Item	Description
num case	The result is the number itself, so <code>rfl</code> closes the proof.
var case	The result is <code>s x</code> , exactly as <code>evalExpr</code> computes, so <code>rfl</code> closes the proof.
add case	The induction hypotheses state that both subexpression results equal <code>evalExpr</code> . <code>simp</code> rewrites and simplifies the expression.

mul case	Same structure as add, but with multiplication.
lt case	The induction hypotheses rewrite both sides, then simp reduces the comparison case.
eq case	The induction hypotheses rewrite both sides, then simp reduces the equality comparison case.

5.2 Main Theorem: Expression Determinism

```

theorem expression determinism :
  forall {e : Expr} {s : State} {v1 v2 : Int},
    EvalExpr e s v1 -> EvalExpr e s v2 -> v1 = v2 := by
  intro e s v1 v2 hEval1 hEval2
  have h1 : v1 = evalExpr e s := evalExpr_sound hEval1
  have h2 : v2 = evalExpr e s := evalExpr_sound hEval2
  exact Eq.trans h1 h2.symm

```

The main proof is short because `evalExpr_sound` does the structural work. The proof receives two derivations: `hEval1` and `hEval2`. Both are evaluations of the same expression `e` in the same state `s`.

Using `evalExpr_sound` on the first derivation gives `h1: v1 = evalExpr e s`. Using it on the second derivation gives `h2: v2 = evalExpr e s`. Since both `v1` and `v2` are equal to the same deterministic expression value, they are equal to each other.

The final line uses transitivity of equality. `h2.symm` reverses `h2` into `evalExpr e s = v2`. Then `Eq.trans h1 h2.symm` proves `v1 = v2`.

6. Statement Semantics Included for Completeness

Although the selected theorem concerns expressions, the Lean file also includes statement syntax and big-step semantics for statements. This makes the Phase 3 file closer to a complete operational semantics model for the selected Hash subset.

6.1 Statement Syntax

```

inductive Stmt where
| assign : Var -> Expr -> Stmt
| seq    : Stmt -> Stmt -> Stmt
| ifElse : Expr -> Stmt -> Stmt -> Stmt
| while  : Expr -> Stmt -> Stmt
deriving Repr

```

`Stmt` models the statement-level subset of Hash. It includes assignment, sequencing, conditional statements, and while-style loops. The Hash for-loop is later treated as syntactic sugar.

6.2 State Update

```

def update (s : State) (x : Var) (v : Int) : State :=
  fun y => if y == x then v else s y

```

The update function is used to model assignment. It returns a new state. In the new state, the updated variable `x` maps to `v`; every other variable keeps its old value from `s`.

6.3 Big-Step Statement Semantics

`BigStep S s s'` means that statement `S` starts in state `s` and finishes in state `s'`. Each constructor corresponds to a statement-level operational semantics rule.

Item	Description
assign	Evaluates the right-hand expression and updates the assigned variable.
seq	Executes <code>S1</code> to an intermediate state, then executes <code>S2</code> from that state.
ifTrue	Executes the then-branch when the condition evaluates to a nonzero integer.
ifFalse	Executes the else-branch when the condition evaluates to 0.
whileFalse	Stops the loop immediately when the condition evaluates to 0.
whileTrue	Executes the loop body once and then repeats the loop from the intermediate state.

6.4 forLoop as Syntactic Sugar

```

def forLoop (init : Stmt) (cond : Expr) (step : Stmt) (body : Stmt) : Stmt :=
  Stmt.seq init (Stmt.while cond (Stmt.seq body step))

```

The Hash `baraye(init; cond; update) { body }` loop is represented as a derived form. It is translated into `init; while(cond) { body; update }`. Therefore, no separate operational rule is required for `forLoop`.

7. Compilation and Expected Result

The file is designed to compile as a single self-contained Lean 4 source file. It does not require Mathlib or any external imports.

Recommended filename: `proof.lean`

```
lean proof.lean
```

Expected behavior: Lean prints no output if all theorems are accepted. In Lean, theorem checking is silent on success. Any type error, missing definition, or invalid proof step would be reported as an error.

Submission note

For the project package, place this file under `Phase3-Proof/proof.lean`. The report should include the selected theorem, the proof idea in natural language, and the complete Lean code or a reference to the submitted proof file.

8. Limitations and Possible Extensions

The proof intentionally focuses on Expression Determinism, which is one of the required theorem options. This makes the proof compact and suitable for presentation, while still demonstrating formal reasoning in Lean.

- The model represents booleans as integers, with `true = 1` and `false = 0`. A richer model could define a separate `Value` type for integers and booleans.
- The statement semantics is included but the final theorem is about expression evaluation only.
- Division, exceptions, `break/continue`, and nested `try/catch` behavior are not modeled in this proof file.
- A future extension could prove full statement determinism or a loop invariant theorem using the `BigStep` relation.

9. Presentation-Ready Explanation

A concise explanation for presentation is:

Oral explanation

In Phase 3, I formally modeled a subset of Hash expressions in Lean 4. I defined both an executable evaluator, `evalExpr`, and a relational big-step evaluator, `EvalExpr`. Then I proved `evalExpr_sound`, which shows that every relational evaluation agrees with the executable evaluator. Finally, I used this result to prove Expression Determinism: if the same expression is evaluated twice in the same state, both evaluations must produce the same value.

Appendix A. Full Lean 4 Source Code

The following is the complete Lean 4 implementation used for Phase 3.

```
/-
Phase 3 - Formal Proof in Lean 4
Chosen theorem: Expression Determinism

Meaning of the theorem:
If an expression e is evaluated in the same state s twice,
and the first evaluation produces v1 while the second produces v2,
then v1 = v2.
-/

namespace ExpressionDeterminismProof

/-- Variable names in Hash are modeled as strings. -/
abbrev Var := String
```

```

/-- A program state maps each variable name to an integer value. -/
abbrev State := Var -> Int

/--
  Expressions of the selected Hash subset.
  Boolean results are represented as integers:
  true  = 1
  false = 0
-/
inductive Expr where
| num : Int -> Expr
| var : Var -> Expr
| add : Expr -> Expr -> Expr
| mul : Expr -> Expr -> Expr
| lt  : Expr -> Expr -> Expr
| eq  : Expr -> Expr -> Expr
deriving Repr

/--
  A deterministic executable evaluator for expressions.
  This is useful as a reference meaning for the relational rules below.
-/
def evalExpr (e : Expr) (s : State) : Int :=
  match e with
  | Expr.num n => n
  | Expr.var x => s x
  | Expr.add e1 e2 => evalExpr e1 s + evalExpr e2 s
  | Expr.mul e1 e2 => evalExpr e1 s * evalExpr e2 s
  | Expr.lt e1 e2 =>
    if evalExpr e1 s < evalExpr e2 s then 1 else 0
  | Expr.eq e1 e2 =>
    if evalExpr e1 s = evalExpr e2 s then 1 else 0

/--
  Relational big-step evaluation for expressions.
  `EvalExpr e s v` means:
  expression `e` evaluates to value `v` in state `s`.
-/
inductive EvalExpr : Expr -> State -> Int -> Prop where
| num : forall (n : Int) (s : State),
  EvalExpr (Expr.num n) s n

| var : forall (x : Var) (s : State),
  EvalExpr (Expr.var x) s (s x)

| add : forall (e1 e2 : Expr) (s : State) (v1 v2 : Int),
  EvalExpr e1 s v1 ->
  EvalExpr e2 s v2 ->
  EvalExpr (Expr.add e1 e2) s (v1 + v2)

| mul : forall (e1 e2 : Expr) (s : State) (v1 v2 : Int),
  EvalExpr e1 s v1 ->
  EvalExpr e2 s v2 ->
  EvalExpr (Expr.mul e1 e2) s (v1 * v2)

| lt : forall (e1 e2 : Expr) (s : State) (v1 v2 : Int),
  EvalExpr e1 s v1 ->
  EvalExpr e2 s v2 ->
  EvalExpr (Expr.lt e1 e2) s (if v1 < v2 then 1 else 0)

| eq : forall (e1 e2 : Expr) (s : State) (v1 v2 : Int),
  EvalExpr e1 s v1 ->
  EvalExpr e2 s v2 ->
  EvalExpr (Expr.eq e1 e2) s (if v1 = v2 then 1 else 0)

/--
  This theorem connects the relational evaluator `EvalExpr`
  to the deterministic function `evalExpr`.

  It says: if the relation says that e evaluates to v,
  then v is exactly the value produced by the function evalExpr.
-/
theorem evalExpr_sound :
  forall {e : Expr} {s : State} {v : Int},
    EvalExpr e s v -> v = evalExpr e s := by
  intro e s v h
  induction h with
  | num n s =>
    rfl
  | var x s =>
    rfl
  | add e1 e2 s v1 v2 h1 h2 ih1 ih2 =>
    simp [evalExpr, ih1, ih2]
  | mul e1 e2 s v1 v2 h1 h2 ih1 ih2 =>
    simp [evalExpr, ih1, ih2]
  | lt e1 e2 s v1 v2 h1 h2 ih1 ih2 =>

```

```

    simp [evalExpr, ih1, ih2]
  | eq e1 e2 s v1 v2 h1 h2 ih1 ih2 =>
    simp [evalExpr, ih1, ih2]

/--
Chosen theorem for Phase 3: Expression Determinism.

If the same expression `e` is evaluated in the same state `s`
and produces `v1` in one derivation and `v2` in another derivation,
then the two results must be equal.
-/
theorem expression_determinism :
  forall {e : Expr} {s : State} {v1 v2 : Int},
    EvalExpr e s v1 -> EvalExpr e s v2 -> v1 = v2 := by
  intro e s v1 v2 hEval1 hEval2
  have h1 : v1 = evalExpr e s := evalExpr_sound hEval1
  have h2 : v2 = evalExpr e s := evalExpr_sound hEval2
  exact Eq.trans h1 h2.symm

/--
Statements of the selected Hash subset.
These are included to complete the operational-semantics model.
-/
inductive Stmt where
  | assign : Var -> Expr -> Stmt
  | seq    : Stmt -> Stmt -> Stmt
  | ifElse : Expr -> Stmt -> Stmt -> Stmt
  | while  : Expr -> Stmt -> Stmt
  deriving Repr

/--
Updating a state after an assignment.
`update s x v` is a new state where x maps to v,
and every other variable keeps its old value from s.
-/
def update (s : State) (x : Var) (v : Int) : State :=
  fun y => if y == x then v else s y

/--
Big-step operational semantics for statements.
`BigStep S s s'` means:
statement `S` starts in state `s` and finishes in state `s'`.
-/
inductive BigStep : Stmt -> State -> State -> Prop where
  | assign : forall (x : Var) (e : Expr) (s : State) (v : Int),
    EvalExpr e s v ->
    BigStep (Stmt.assign x e) s (update s x v)

  | seq : forall (S1 S2 : Stmt) (s smid sout : State),
    BigStep S1 s smid ->
    BigStep S2 smid sout ->
    BigStep (Stmt.seq S1 S2) s sout

  | ifTrue : forall (b : Expr) (S1 S2 : Stmt) (s sout : State) (v : Int),
    EvalExpr b s v ->
    v ≠ 0 ->
    BigStep S1 s sout ->
    BigStep (Stmt.ifElse b S1 S2) s sout

  | ifFalse : forall (b : Expr) (S1 S2 : Stmt) (s sout : State),
    EvalExpr b s 0 ->
    BigStep S2 s sout ->
    BigStep (Stmt.ifElse b S1 S2) s sout

  | whileFalse : forall (b : Expr) (S : Stmt) (s : State),
    EvalExpr b s 0 ->
    BigStep (Stmt.while b S) s s

  | whileTrue : forall (b : Expr) (S : Stmt) (s smid sout : State) (v : Int),
    EvalExpr b s v ->
    v ≠ 0 ->
    BigStep S s smid ->
    BigStep (Stmt.while b S) smid sout ->
    BigStep (Stmt.while b S) s sout

/--
The Hash `baraye(init; cond; update) { body }` loop can be treated
as syntactic sugar for:
init; while(cond) { body; update }
-/
def forLoop (init : Stmt) (cond : Expr) (step : Stmt) (body : Stmt) : Stmt :=
  Stmt.seq init (Stmt.while cond (Stmt.seq body step))

end ExpressionDeterminismProof

```